

# Boundary-Mutation Testing for Pattern-Based Secret Detection: A Rule-Level Method and Cross-Scanner Evaluation

Shweta Mishra<sup>1\*†</sup>

<sup>1\*</sup>Independent Researcher, Moradabad, India.

Corresponding author(s). E-mail(s): [shweta.mishra.research@gmail.com](mailto:shweta.mishra.research@gmail.com);

<sup>†</sup>ORCID: 0009-0002-8443-1678

## Abstract

Pattern-based secret scanners are commonly validated with example-based fixtures that fix one variable: the text surrounding a credential. We introduce boundary-mutation testing to vary that context, generating credentials from each rule’s own regular expression, embedding them in realistic source contexts, and classifying outcomes at the rule level rather than the tool level, yielding three detection metrics. Applied to three scanners — a 43-rule open-source scanner, Gitleaks 8.21.2, and TruffleHog 3.82.13 — detection in the primary subject holds at  $\geq 0.9976$  across ten contexts but collapses to 0.5233 when a credential ends in a hyphen. Five rules are affected: two, with fixed-count quantifiers, fail totally and deterministically; three, with variable-count quantifiers, backtrack and match a truncated credential; an entropy fallback rescues some failures but downgrades their severity. We validate a repair restoring full robustness with no new false positives; a plausible first attempt silently regressed two rules, caught only by re-running the battery. Gitleaks has an unrelated, source-confirmed defect — a hard-coded terminator allowlist causing total misses for most credential types — while TruffleHog shows no boundary fragility but narrowest coverage. We report marginal, not conditional, failure probabilities: one common token format is structurally immune, another fails once in 64; on 292,527 lines of real code, the false-positive ordering inverts relative to the synthetic corpus. Because per-type detection is near-deterministic, the comparison unit is ten credential types, not hundreds of samples; no recall difference reaches significance, so we report the null result, not a ranking.

**Keywords:** secret detection, credential leakage, mutation testing, metamorphic testing, regular expressions, empirical software engineering

# 1 Introduction

Credential leakage through source repositories is a persistent and largely preventable security failure (Meli et al. 2019). The standard control is a pattern-based secret scanner: a table of regular expressions, usually paired with an entropy heuristic, applied to diffs or working trees. Gitleaks (Gitleaks contributors 2024), TruffleHog (Truffle Security Co. 2024), GitHub’s hosted secret scanning, and the scanner studied here all belong to this family.

Such rule tables are commonly validated with example-based fixtures. A contributor adding a rule writes one or two credentials of the new shape into a test file and asserts they are detected; the scanner we study validates 43 rules with a 313-line fixture suite, and the practice is visible in the test suites of the other two tools as well. We do not claim to have surveyed the field exhaustively, and we do not need to: the weakness we identify follows from the structure of example-based fixtures, not from their prevalence.

That weakness is this. A regular expression does not match a credential in isolation; it matches a credential *together with its surroundings*, and anchoring assertions such as `\b` take effect precisely at the join between the two. Hand-written fixtures almost always place the credential in one canonical position — inside a double-quoted assignment, ending in an alphanumeric character. A rule that fails only when a credential is passed as a bare function argument, or only when its final character happens to be a hyphen, cannot be observed by a suite in which no fixture is written that way.

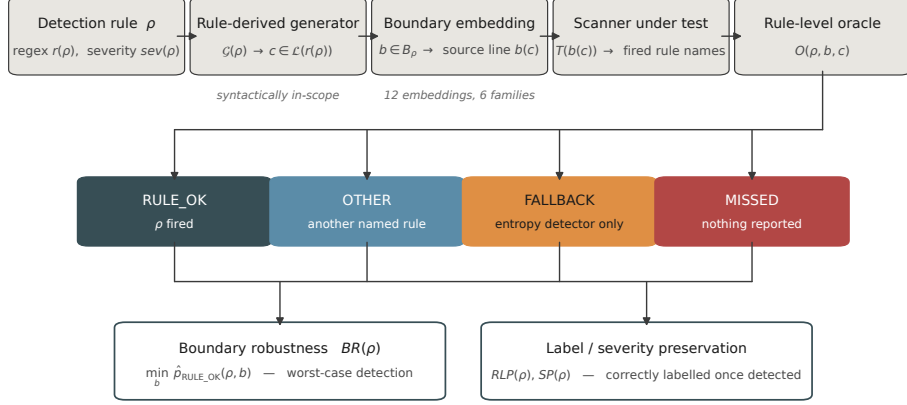
This paper introduces *boundary-mutation testing* to vary that surrounding context systematically. Three design choices distinguish it from writing more test cases. First, credentials are generated from *each rule’s own regular expression*, so the corpus is derived from the artefact under test rather than from the experimenter’s intuitions. Second, each credential is embedded in a battery of contexts, each one a way a credential legitimately appears in real source. Third — and this is what makes the defects visible — outcomes are classified at the *rule* level, distinguishing “the credential’s own rule fired” from “something fired” (Fig. 1). The gap between those two is where a severity downgrade hides, and a tool-level binary metric cannot see it.

We apply the method to three independent scanners to test *transferability and failure-mode diversity*, not to produce a statistically powered ranking; Section 11.1 shows the latter is not supported by ten shared credential types regardless of method. It transfers, and it exposes a distinct scanner-specific behaviour in each: a defect in the first two, and in the third a boundary-robust detector whose limits lie elsewhere. We report the mechanism behind each, the marginal (not merely conditional) probability that each affects a real credential, and a comparative evaluation in which we take some care to avoid disadvantaging a tool whose architecture differs from the others’.

## Contributions.

1. **A method and three metrics.** Boundary-mutation testing, formally defined as a family of metamorphic relations (Section 5), with *boundary robustness*  $BR(\rho)$ , *rule-label preservation*  $RLP(\rho)$  and *severity preservation*  $SP(\rho)$  derived from its outcome classification.

Boundary-mutation testing: credentials are generated from the rule under test, embedded in contexts a real credential occupies, and judged at the rule level; two metrics summarise the resulting outcome distribution



**Fig. 1** Boundary-mutation testing. A credential is generated from the rule under test, embedded in a context that a real credential of that type occupies, and the outcome is classified at the rule level. The four outcome classes yield the three metrics of Section 5.2.

2. **Evidence that it generalises.** The same battery applied to three scanners exposes three distinct behavioural profiles (Section 10), including a terminator-allowlist defect in Gitleaks that, to the best of our knowledge, has not been previously reported, affecting six of the nine credential types it detects.
3. **Mechanisms, not just findings.** Each defect is traced to a specific regular-expression construct and confirmed on minimal pairs, with marginal failure probabilities computed under an explicitly stated model (Section 8).
4. **A validated repair, and a caution about validating repairs.** We patch the five defective rules, re-run the full battery, and confirm  $BR = 1.0$  with no new false positives on real code. A first candidate patch, plausible on inspection, is shown by the same battery to regress two rules to total failure on a case the original rule handled correctly; we report both results as evidence that a proposed regex repair is a hypothesis, not a fact, until it is run (Section 9).
5. **Benchmark design as a measured confound.** Corpus construction moves one tool’s recall by +0.10; deployment condition moves another’s detection by +0.33; and the real-code false-positive ordering inverts relative to the synthetic corpus (Sections 11–12). A paired analysis at the correct unit of observation shows that none of the recall differences we measure is statistically significant, so we report the comparison as a characterisation of trade-offs and not as a ranking.

**Scope.** This is a methods paper about rule-level detection. Two subsystems of the primary subject are therefore deliberately excluded from the main study. Its LLM layer is unmeasured because no funded provider credential was available (Section 5.5). Its operational characteristics — queueing, concurrency, and fault tolerance — were

separately characterised and are reported in Online Resource 1; they are outside the scope of the research questions and bear on none of the claims made here.

**Research contribution versus engineering artefact.** Because one of the three evaluated systems is the author’s own, we state this distinction explicitly rather than leave it implicit. The *research contribution* is boundary-mutation testing itself: the method, its formalisation as a family of metamorphic relations, its three derived metrics, and the empirical evidence in Sections 7–9 about boundary-sensitive detector behaviour, obtained by applying that method to three independently developed systems. GitHub Autopilot is an *engineering artefact* used to instantiate and validate the methodology — one experimental subject among three, not the paper’s object of study. Nothing in our claims depends on which of the three subjects it is; Section 5.6 details the controls taken because it happens to be ours.

## 2 Research Questions

**RQ1** *Does rule-level detection depend on the source context surrounding a credential, and which contexts degrade it?*

**RQ2** *What mechanism produces any context sensitivity found, and what is the marginal probability that it affects a real credential?*

**RQ3** *Does the method transfer to independently developed scanners, and is the defect class shared or tool-specific?*

**RQ4** *How do the three scanners compare on identical corpora, and how much do corpus construction and deployment condition affect that comparison?*

**RQ5** *What is the false-positive behaviour of each scanner on the evaluated corpus of unmodified real open-source code?*

## 3 Related Work

**Secret detection.** Meli et al. (2019) measured secret leakage on GitHub at scale, establishing that leaked credentials are common and long-lived. Subsequent work targets the false-positive burden that impedes adoption (Saha et al. 2020; Rahman et al. 2019), and recent studies examine the accuracy of detection tooling in practice (Basak et al. 2022).

Two lines are especially close to ours and warrant an explicit comparison. Basak et al. (2023a) evaluate nine open-source and proprietary secret detection tools against a benchmark corpus and report wide disagreement between them, particularly in precision; Basak et al. (2023b) contribute the benchmark dataset that makes such comparisons possible, drawn from secrets checked into public repositories. More recently, Li et al. (2025) evaluate checked-in secret detection for Android applications and likewise conclude that current techniques have substantial limitations. All three establish, from independent evidence, the premise this paper takes as its starting point: detection tools disagree, and their measured accuracy is sensitive to what they are measured against.

Our question is different in what it holds fixed. Those studies compare tools on a corpus of credentials collected or curated from the field, so the credential and its surrounding text vary together and the tool is the manipulated variable. We invert that. The credential is generated from the detector’s own rule, and the *context* surrounding it is the manipulated variable, so a change in outcome is attributable to the boundary rather than to the sample. Outcomes are then judged against a rule-level oracle rather than a tool-level verdict, which separates a miss from a detection carrying the wrong label and severity. That combination — inputs generated from each artefact’s own declared language, context as the treatment, and a rule-level oracle — is what we have not found in prior work, and it is what allows the defects in Section 10 to be traced to a specific regular-expression construct rather than reported as an aggregate accuracy difference. To the best of our knowledge, no prior study systematically varies the context surrounding a credential while attributing outcomes at the rule level, and we are not aware of prior reports of either defect in Section 10. Table 1 positions this work against the closest related lines.

**Mutation and metamorphic testing.** Classical mutation testing perturbs the *program* to assess a test suite (Jia and Harman 2011; DeMillo et al. 1978). We invert this: the artefact under test is a rule table, and we perturb the *input*. In that respect the method is metamorphic (Chen et al. 2018; Segura et al. 2016): our context mutations are metamorphic relations, since re-quoting a credential or moving it into a YAML file should not change whether it is a credential. The distinguishing elements here are that inputs are generated *from the artefact’s own rules*, and that the oracle is rule-level rather than output-level.

**Input generation and grammar-based testing.** Generating inputs from a formal description relates to grammar-based fuzzing (Godefroid et al. 2008) and property-based testing (Claessen and Hughes 2000), where a generator produces values satisfying a specification. Our generator consumes the detector’s own regular expressions, which makes the positive corpus definitionally in-scope for the rule being tested — a miss cannot be dismissed as outside the rule’s syntactic specification, though, as Section 5.1 sets out, syntactic scope is not the same as provider-semantic validity. Work on regular-expression correctness and behaviour (Chapman and Stolee 2016; Michael et al. 2019) documents that developers frequently misunderstand anchoring and character-class semantics; both defects we report are instances of exactly that, in security-critical rules.

**Benchmark design as a confound.** Evidence continues to accumulate that benchmark construction, not only technique quality, drives measured outcomes (Liem and Panichella 2020; Herbold et al. 2020). Sections 11–12 contribute three instances in which construction alone reorders tools.

**Evaluating security scanners.** A recurring finding in the evaluation of static security tools is that measured effectiveness is highly sensitive to the benchmark rather than to the tool: reported detection rates vary widely across corpora, and tools tuned to one benchmark generalise poorly to another. Our Sections 11–12 are three instances of the same phenomenon in the secret-detection setting, with the additional observation that the sensitivity is large enough to reverse the ordering. Where that literature

**Table 1** This work against the closest related testing methods.

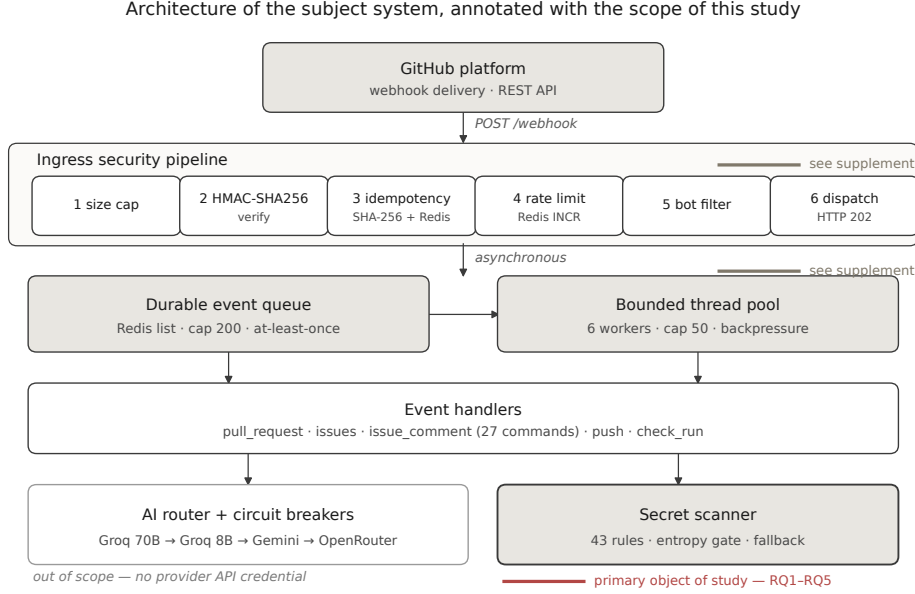
| Line of work                                                                                                                                                                 | Scan. | Mut. | Meta. | Gen. | Oracle |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|------|-------|------|--------|
| Secret-leakage & FP studies ( <a href="#">Meli et al. 2019</a> ; <a href="#">Saha et al. 2020</a> ; <a href="#">Rahman et al. 2019</a> ; <a href="#">Basak et al. 2022</a> ) | ✓     |      |       |      |        |
| Comparative scanner evaluation on curated corpora ( <a href="#">Basak et al. 2023a,b</a> ; <a href="#">Li et al. 2025</a> )                                                  | ✓     |      |       |      |        |
| Classical mutation testing ( <a href="#">Jia and Harman 2011</a> ; <a href="#">DeMillo et al. 1978</a> )                                                                     |       | ✓    |       |      |        |
| Metamorphic testing ( <a href="#">Chen et al. 2018</a> ; <a href="#">Segura et al. 2016</a> )                                                                                |       |      | ✓     |      |        |
| Grammar-/property-based generation ( <a href="#">Godefroid et al. 2008</a> ; <a href="#">Claessen and Hughes 2000</a> )                                                      |       |      |       | ✓*   |        |
| <b>This work</b>                                                                                                                                                             | ✓     | ✓    | ✓     | ✓    | ✓      |

Scan. = evaluates a security scanner; Mut. = mutation testing; Meta. = metamorphic testing; Gen. = inputs generated from a formal rule/specification; Oracle = outcome judged at rule level. \*Generates from a formal specification, not from a detector’s own rule. Regex-correctness studies ([Chapman and Stolee 2016](#); [Michael et al. 2019](#)) motivate Sections 8–10 but are an empirical study, not a method, so are cited in text only and omitted from the rows above.

typically compares tools, we use the sensitivity itself as the object of study and decline to produce a ranking.

**The unit of analysis in tool comparison.** Our Section 11.1 rests on a point that is well established in empirical software engineering but easy to lose in tool evaluation: when outcomes are deterministic within a stratum, the stratum and not the individual sample is the independent observation. Treating 400 credentials drawn from 10 deterministic types as 400 observations inflates the evidence by the number of samples per type. We report the paired test at the type level for this reason, and consequently report a null result where the sample-level intervals would have suggested a clear ordering.

**Standards and formats.** The defects reported here affect detection of JSON Web Tokens ([Jones et al. 2015](#)) and OAuth bearer credentials ([Hardt 2012](#)), whose encodings determine the marginal probabilities of Section 8; the HMAC construction underlying JWT signatures is specified in ([Krawczyk et al. 1997](#)). Complexity ([McCabe 1976](#)) and coverage ([Zhu et al. 1997](#)) appear only in the supplementary subject characterisation, and the resilience patterns exercised there are due to [Nygard \(2018\)](#) and [Fowler \(2014\)](#). [Wessel et al. \(2018\)](#) and [Vasilescu et al. \(2015\)](#) study bots and CI at the process level; our subject is one such bot, but it is studied here only as the owner of a rule table.



**Fig. 2** The primary subject, annotated with the scope of this study. Only the secret scanner is the object of the research questions; the ingress and concurrency subsystems are characterised in the supplementary artefact, and the LLM routing layer is unmeasured.

## 4 Subject Systems

The unit of analysis throughout is a *detection rule*, not a tool. We study three rule tables belonging to independently developed scanners.

The primary subject is the secret scanner of GitHub Autopilot, an open-source self-hosted GitHub App available at <https://github.com/Shweta-Mishra-ai/github-autopilot> and studied here at commit [38b2013](#). Its scanner comprises 43 regular-expression rules with per-rule severities, plus one unanchored entropy fallback that fires on high-entropy strings no named rule claims. We make no claim about its deployment scale; it is studied because its rule table is open, small enough to characterise exhaustively, and structurally representative of the regex-plus-entropy family.

Fig. 2 places that scanner within the system it belongs to, and marks the scope of this study. The scanner is the primary object of study. The ingress pipeline, queue and worker pool are characterised in the supplementary artefact but are not the subject of any claim made here. The LLM routing layer is unmeasured entirely (Section 5.5).

Table 2 characterises the subject quantitatively. These measurements describe *what was evaluated*; they are not evidence for or against the correctness of its scanner, and we do not treat them as results. They matter for two reasons only. First, they establish that the subject is a maintained system with a real test suite rather than a toy, which is what makes a defect in its rule table interesting. Second, they establish that its rule

**Table 2** Characteristics of the primary subject at commit 38b2013. Provided to characterise the experimental subject; not treated as evidence of scanner correctness.

| Property                        | Value                         |
|---------------------------------|-------------------------------|
| Application modules             | 91                            |
| Application lines of Python     | 19,352                        |
| Test lines of Python            | 19,825                        |
| Functions and methods           | 617                           |
| Mean cyclomatic complexity      | 4.67                          |
| Maximum cyclomatic complexity   | 34                            |
| Test cases                      | 2,054                         |
| Test outcome                    | 2,054 passing                 |
| Statement coverage              | 83.9% (7,600 statements)      |
| Lint violations ( <b>ruff</b> ) | 0                             |
| Scanner rules                   | 43 named + 1 entropy fallback |
| Rules analysed in this study    | 24 (Section 5)                |
| Scanner fixture suite           | 313 lines                     |

table is small enough to characterise exhaustively — all 43 rules were examined, not a sample.

The last two rows are the ones that motivate this paper. A 43-rule table validated by a 313-line fixture suite is not unusual, and it is not negligent; it is the normal ratio in this class of tool. Section 7 shows what that ratio leaves unobserved.

The comparison subjects are Gitleaks 8.21.2 and TruffleHog 3.82.13, both widely used in practice and both independently developed by parties unconnected to the primary subject. Gitleaks is rule-table-driven with a public TOML configuration, which makes source-level mechanism confirmation possible; TruffleHog is detector-driven with co-occurrence and live-verification logic, an architecture different enough from the other two that Section 11 treats it as a distinct evaluation condition rather than a like-for-like competitor.

## 5 Method: Boundary-Mutation Testing

This section defines each element of the method summarised in Fig. 1.

### 5.1 Definition

Let  $\rho$  be a detection rule with regular expression  $r(\rho)$  and declared severity  $sev(\rho)$ , and let  $\mathcal{G}(\rho)$  be a generator producing strings in  $\mathcal{L}(r(\rho))$ , the language of that expression. Let  $B$  be a set of *boundary embeddings*, each  $b \in B$  a function mapping a credential  $c$  to a line of source text  $b(c)$  in which  $c$  appears as a real credential would. Let  $T$  be the tool under test, returning a set of fired rule names  $T(b(c))$ .



## Syntactic versus semantic validity

Two notions of “valid credential” must be kept apart, because the method guarantees one and not the other. Write  $\mathcal{L}(r(\rho))$  for the set of strings the rule’s own expression accepts, and  $C_\rho$  for the set of strings the issuing provider would actually mint as a credential of that type. In general  $C_\rho \subseteq \mathcal{L}(r(\rho))$ , and the inclusion is strict: a rule that accepts SG, followed by 22 and then 43 characters from a permitted class also accepts strings that SendGrid would never issue.

Our generator samples from  $\mathcal{L}(r(\rho))$ , so it guarantees *syntactic* in-scopeness only. This is the precise sense in which a miss is a defect: the rule’s author declared, by writing  $r(\rho)$ , that strings in  $\mathcal{L}(r(\rho))$  occurring in these contexts are to be reported, and the tool did not report one. That is an internal inconsistency in the rule, and it is what the method detects. It is *not* a claim that the specific string would ever be issued. Where  $C_\rho$  is much smaller than  $\mathcal{L}(r(\rho))$ , a defect found on  $\mathcal{L}(r(\rho)) \setminus C_\rho$  may have no field consequence, which is exactly why Section 8 computes marginal probabilities under an explicit issuance model rather than resting on the generated rates alone. Where a provider’s format is public and the rule encodes it faithfully — the case for the five defective rules we report — the two sets nearly coincide and the distinction is immaterial.

## The metamorphic relation

The mutations are metamorphic relations over the tool’s input. For a credential  $c$  and two embeddings  $b_i, b_j$  both applicable to  $\rho$ :

$$MR(c, b_i, b_j) : \quad T(b_i(c)) \ni \rho \iff T(b_j(c)) \ni \rho.$$

The invariant asserted is not textual equality between  $b_i(c)$  and  $b_j(c)$  — they differ by construction — but *preservation of credential detectability under a semantics-preserving change of surrounding context*. Re-quoting a credential, moving it into a YAML file, or passing it as a call argument does not change whether the string is a credential, so a rule whose verdict changes has violated the relation. A violated relation localises the defect without requiring an independent oracle for what the tool “should” output, which is the standard advantage of metamorphic testing and the reason it applies here: there is no ground-truth detector to compare against.

For a credential  $c \sim \mathcal{G}(\rho)$  and embedding  $b$ , the outcome is classified as

$$O(\rho, b, c) = \begin{cases} \text{RULE\_OK} & \rho \in T(b(c)) \\ \text{OTHER} & T(b(c)) \setminus (F \cup \{\rho\}) \neq \emptyset \\ \text{FALLBACK} & \emptyset \neq T(b(c)) \subseteq F \\ \text{MISSED} & T(b(c)) = \emptyset \end{cases}$$

where  $F$  is the tool’s set of unanchored fallback detectors. An embedding  $b$  is *applicable* to  $\rho$  when  $b$  preserves membership in  $\mathcal{L}(r(\rho))$ ; forcing a final character the rule’s own character class excludes would produce a string that is no longer a credential of that

type, and a miss there would be correct behaviour rather than a defect. Let  $B_\rho \subseteq B$  be the applicable embeddings.

## 5.2 Metrics

Writing  $\hat{p}_x(\rho, b)$  for the empirical rate of outcome  $x$  over  $n$  samples, we define three metrics. The first summarises how often the rule works; the other two summarise, of the credentials that were detected at all, how faithfully they were reported.

*Boundary robustness* is the rule’s worst-case behaviour over contexts in which it is supposed to work:

$$BR(\rho) = \min_{b \in B_\rho} \hat{p}_{\text{RULE\_OK}}(\rho, b). \quad (1)$$

*Rule-label preservation* is the fraction of detected credentials reported under the credential’s own rule:

$$RLP(\rho) = \frac{\sum_{b \in B_\rho} \hat{p}_{\text{RULE\_OK}}(\rho, b)}{\sum_{b \in B_\rho} (1 - \hat{p}_{\text{MISSED}}(\rho, b))}. \quad (2)$$

*RLP* measures rule *identity*, which is not the same thing as severity. A tool that reports a credential under a different rule carrying the same severity has mislabelled it without downgrading it, and a metric named for severity should not penalise that. We therefore define severity preservation separately, over the severity actually reported. Let  $\widehat{sev}$  denote the severity attached to the finding a tool emits:  $sev(\rho)$  for *RULE\_OK*,  $sev(\rho')$  for an *OTHER* outcome firing  $\rho'$ , and  $sev(F)$  for a *FALLBACK* outcome. Then

$$SP(\rho) = \frac{\sum_{b \in B_\rho} \mathbb{E}[\mathbb{1}\{\widehat{sev} = sev(\rho)\}]}{\sum_{b \in B_\rho} (1 - \hat{p}_{\text{MISSED}}(\rho, b))}. \quad (3)$$

$SP < 1$  means the tool is quietly reporting findings it did detect at the wrong severity;  $RLP < 1$  means it is attributing them to the wrong rule. The two coincide only when no substitute label shares the true rule’s severity. A rule is *boundary-fragile* when some applicable embedding degrades it materially relative to the canonical one.

Both metrics require a severity oracle. For the primary subject we have one: each rule declares a severity, and the fallback detector’s severity is fixed at *MEDIUM* in source. Section 7 reports both. For the two external scanners we do not attempt a severity mapping — their severity models are not commensurable with the subject’s, and inventing a correspondence would manufacture a comparison the data does not support — so Section 10 reports *BR* and *RLP* only.

## 5.3 Instantiation

$\mathcal{G}$  is implemented with **exrex**, which enumerates strings from a regular expression. Of the 43 rules, seven are structural (PEM headers, connection-string schemes) and twelve are keyword-anchored — they require an adjacent keyword such as **aws.secret**, so a neutral embedding would make them unmatchable by construction — leaving 24 rules. We use  $n = 120$  samples per cell.

**Table 3** The boundary battery: six families, twelve embeddings

| Family          | Embedding        | Mechanism exercised                      |
|-----------------|------------------|------------------------------------------|
| Assignment      | quoted           | canonical fixture shape                  |
|                 | single-quoted    | alternate quote character                |
|                 | unquoted         | no delimiter at all                      |
| Structured data | YAML value       | configuration syntax                     |
|                 | JSON value       | configuration syntax                     |
|                 | dotenv           | environment-file syntax                  |
| Expression      | call argument    | credential passed programmatically       |
| Collection      | list member      | separator-terminated                     |
| URL             | query parameter  | the shape credentials most often leak in |
| Lexical         | extra whitespace | whitespace as terminator                 |
| Final character | ends -           | permitted terminal, word character       |
|                 | ends -           | permitted terminal, non-word character   |

## Constructing the boundary battery

$B$  comprises twelve embeddings drawn from six families (Table 3). The families are not arbitrary: each corresponds to a distinct syntactic mechanism by which a character can come to sit immediately adjacent to a credential in source code, which is the only thing an anchoring assertion can observe. Within a family the members vary the adjacent character while holding the mechanism fixed.

We make no claim that twelve embeddings exhaust the boundary space, and Section 14 names the family we know is missing. The battery is a *representative* boundary battery, chosen to cover each mechanism at least once, not an exhaustive enumeration. Its adequacy is argued from mechanism coverage, and is supported by the distinct scanner-specific behaviours it exposed across the three tools to which it was applied.

## Freezing the expected rule for external scanners

Judging one tool by another project’s rule names would be unfair, so for Gitleaks and TruffleHog the expected rule per credential type is established empirically and frozen before any mutation is applied. For each credential type we (i) generate samples from the primary subject’s rule for that type; (ii) present them in the *canonical* embedding only; (iii) record which `RuleID` or `DetectorName` the tool fires; (iv) require that the

same identifier fire on every one of the canonical samples, so that the expectation is not set by a single observation; and (v) freeze that identifier as the expected rule for all subsequent embeddings. A type on which a tool fires no rule, or fires inconsistently, in the canonical context is excluded from that tool’s aggregate and reported separately as a coverage gap, so that missing coverage is never scored as fragility.

The residual risk in this procedure is that the canonical context itself provokes a wrong-but-consistent attribution, which would then be frozen as the expectation. We checked the frozen identifiers by inspection and they are semantically correct in every case (for instance `aws-access-token` for AWS keys); no type was attributed to a generic or unrelated detector. The procedure cannot detect an error that is both consistent and semantically plausible, which we note as a limitation rather than claim to have eliminated.

## 5.4 A control that proved necessary

An initial harness used the identifier `SECRET` in its embeddings and measured a uniform 100% detection rate, apparently exonerating every rule. The identifier was itself triggering the scanner’s *generic* keyword rules, which fired on the surrounding text and masked the failure of the specific rule under test. All results below use deliberately neutral identifiers (`cfg_value`). We report this because it is the failure mode a replication is most likely to reproduce, and because it is a concrete instance of why the rule-level oracle matters: a tool-level oracle could not have distinguished the two situations at all.

## 5.5 What we did not measure

The primary subject ships an AI-output evaluation harness requiring a funded provider key; our environment had neither the key nor outbound access to the four providers. We therefore do not measure LLM latency, output quality, or hallucination rate, and substitute no estimates. Nothing in this paper speaks to the subject’s AI behaviour.

We also do not treat the subject’s operational behaviour as a result of this study. Its queueing, concurrency and fault-tolerance characteristics were measured — including a lost-update oracle, a synchronised-start scaling curve, a payload sweep and a fault-injection battery — and are reported in Online Resource 1, with the full raw data in the replication package. They appear as a separate online resource rather than as a research question because they concern a different artefact (a running service) than the one this paper is about (a rule table), and because no claim in this paper depends on them.

## 5.6 Researcher bias and evaluation controls

The author wrote the primary subject. That is a competing interest with a specific and predictable direction: it would bias the study toward flattering the subject’s scanner. We state the controls used against it, and the evidence that they took effect.

Procedurally: the subject was frozen at commit `38b2013` before any benchmark was run and never modified in response to a result; the external tools are unmodified official release binaries at pinned versions, verified by SHA-256; all three tools receive

byte-identical corpora and identical execution conditions; every raw output is retained in the replication package, including the runs that contradict the subject; no finding was excluded after inspection; and no defect found in the subject was fixed before measurement.

Substantively, three of this paper’s results are unfavourable to the subject and are reported at full strength. The paper’s single largest finding is a defect in the subject’s own scanner, not in a competitor’s. The subject’s rules are characterised as *shallower* than Gitleaks’ — prefix, class and length only — and that shallowness is identified as the cause both of its perfect recall and of its 17 false positives (Section 11). Every one of the subject’s ten real-world findings is a false positive, itemised in Section 12. Most consequentially, the paired analysis in Section 11 finds that *none* of the recall differences favouring the subject is statistically significant, and we therefore withdraw any ranking claim rather than report the raw ordering as a result.

The design that most constrains this bias is the one built into the method: credentials are generated from *each tool’s own* declared rules, so the corpus cannot be tuned to favour a particular tool’s shapes without that tuning being visible in the rule table itself.

## 5.7 Use of generative AI

The author used generative AI assistance during this work, under the author’s direction and review. Anthropic’s Claude, accessed through its command-line coding interface, was used to draft and revise manuscript prose from the author’s outlines and decisions, to implement the benchmark harnesses and figure-generation code to the author’s specification, to debug those harnesses, and to support diagnostic analysis when tracing observed failures to specific regular-expression constructs. OpenAI’s ChatGPT was used separately to review manuscript drafts and to cross-check the submission against the journal’s author guidelines; its suggestions were evaluated by the author and adopted selectively, and several were verified against primary sources and rejected as incorrect.

The author conceived the study, chose its research questions, experimental design and unit of analysis, executed every experiment, inspected the raw outputs, verified each numeric claim against the recorded data, and takes full responsibility for the design, the analyses, the interpretations, and the final manuscript. No AI system is an author of this work, and none is credited as one: authorship requires accountability for the claims made, which an AI system cannot hold. Every generated artefact was reviewed before use, and every number reported here is re-derived from raw output by a verification script included in the replication package rather than taken on trust from generated text.

# 6 Experimental Setup

## 6.1 Environment

All measurements were produced in a single containerised Linux environment running CPython 3.11.15. The comparison scanners are official release binaries, Gitleaks

8.21.2 and TruffleHog 3.82.13, pinned by version and verified by SHA-256 digest at reproduction time; the digests are recorded in the replication package. TruffleHog is invoked with `--no-verification`, since live credential verification requires outbound network access to the issuing providers and was not available. Where a Redis instance is required it is `redis-server` 7.0.15 bound to loopback with persistence disabled.

Hardware is deliberately not reported as a study parameter. Every result in Sections 7–12 is a detection *outcome* — which rule fired on which input — and is therefore invariant to the machine that produced it. The only hardware-sensitive measurements in this work are the operational timings, which are reported in the supplementary artefact and are not the basis of any claim here.

## 6.2 Corpora and sampling

Three corpora are used, and they differ deliberately.

The *mutation corpus* (Sections 7–10) is generated per rule from that rule’s own regular expression. It uses  $n = 120$  samples for each of the 24 analysed rules in each of the 12 embeddings. After inapplicable combinations are excluded, 251 rule–context cells remain populated.

The *comparative corpora* (Section 11) hold the negatives fixed at 700: 500 lines of real source drawn from the subject, and 200 structurally credential-shaped non-secrets. Only the construction of the 400 positives varies. Those are 40 samples for each of the 10 credential types shared by all three scanners.

The *real-code corpus* (Section 12) is 292,527 lines of unmodified source from four open-source projects. Each is pinned by commit in the replication package, and scanned line by line as added diff lines.

## 6.3 Determinism and seeding

The mutation and comparative harnesses are seeded (20260823) and reproduce identically; we verified this directly by re-running each and diffing every recorded field except wall-clock time, and `verify_paper.py` in the replication package re-derives every numeric claim in this manuscript from the raw output on every reproduction run. This verification is not decorative. An earlier version of the comparative harness recorded that seed in its output while drawing all credential material from `secrets`, which is a CSPRNG and ignores it; the corpus was therefore different on every run, and two reported figures did not reproduce. The defect was found by running the package’s own reproduction script end to end rather than by inspection, and is the second seeding defect this work has had to correct in its own infrastructure (Section 14 records the first). The harness now draws from the seeded generator and its output is byte-identical across runs apart from timings. The cross-tool harness is *not* seeded: it draws credential material from `secrets.SystemRandom`. We report this rather than retrofit a seed. The property it establishes is that a rule fails on *every* hyphen-terminated credential, not on particular ones. That is a claim about the whole class, and fixing one draw would not strengthen it. Its conclusions were stable across runs, and the per-type outcomes it produces are deterministic in the sense established in Section 11.1: every credential type yields a rate of exactly 0.000 or exactly 1.000.

**Table 4** Aggregate outcome by context (24 rules,  $n=120$  per cell)

| Context           | $n$        | Rule fires    | Fallback      | Missed        |
|-------------------|------------|---------------|---------------|---------------|
| quoted assignment | 2880       | 0.9990        | 0.0003        | 0.0007        |
| single-quoted     | 2880       | 0.9997        | 0.0003        | 0.0000        |
| unquoted          | 2880       | 0.9986        | 0.0000        | 0.0014        |
| YAML value        | 2880       | 0.9983        | 0.0000        | 0.0017        |
| JSON value        | 2880       | 0.9986        | 0.0014        | 0.0000        |
| dotenv            | 2880       | 0.9976        | 0.0000        | 0.0024        |
| URL query         | 2880       | 0.9979        | 0.0000        | 0.0021        |
| list member       | 2880       | 0.9990        | 0.0000        | 0.0010        |
| call argument     | 2880       | 0.9993        | 0.0000        | 0.0007        |
| extra whitespace  | 2880       | 0.9986        | 0.0003        | 0.0010        |
| ends in _         | 720        | 1.0000        | 0.0000        | 0.0000        |
| <b>ends in -</b>  | <b>600</b> | <b>0.5233</b> | <b>0.2317</b> | <b>0.2450</b> |

## 6.4 Statistical treatment

Proportions are reported with Wilson 95% confidence intervals. These behave correctly at the boundary values of 0 and 1, which occur throughout this data. Between-tool comparisons are *not* tested at the sample level. Per-type detection is near-deterministic — fifty-eight of sixty tool-by-type cells are exactly 0 or 1 — so samples within a credential type are very close to repetitions rather than independent observations. The effective unit of analysis is therefore the credential type. Section 11.1 sets out the consequences and reports an exact paired sign test over the ten types. We report effect sizes and intervals in preference to  $p$ -values throughout, and where we do test, we report the discordant cases themselves rather than a statistic alone.

## 7 RQ1: Context Sensitivity

Fig. 3 gives the full rule  $\times$  context matrix and Table 4 the aggregate, both from a seeded run (20260823) that reproduces exactly; we confirmed this by re-running the harness and diffing every field but wall-clock time. Across ten of twelve contexts the rule-level detection rate never falls below 0.9976. Forcing the final character to an underscore is likewise harmless (1.0000). One context breaks it: when the final character is a hyphen — which the affected rules’ own character classes permit — the aggregate falls to **0.5233**, with 23.2% of samples rescued only by the entropy fallback and **24.5%** reported by nothing at all.

### 7.1 Reproducibility and repeated-run robustness

Two of the five fragile rules are deterministic in the strongest sense available to this method: *GCP API Key* and *SendGrid API Key* score  $BR = 0.0000$  in *every one* of ten independently seeded repetitions ( $n = 120$  each), with zero variance. Both are governed by fixed-count quantifiers ( $\{35\}$ ,  $\{43\}$ ), and their failure depends only on

whether the credential ends in a hyphen, not on any other property of the generated string.

The remaining three fragile rules — governed by variable-count quantifiers ( $\{50,\}$ ,  $\{68,\}$ ,  $\{10,\}$ ) — do not share this property. Across the same ten repetitions their truncated-match rate ranges from 92.5% to 98.3% (pooled mean 95.3%), compared with 84–89% in the single reproducible run reported in Table 4 and Table 5. This is not measurement noise in the ordinary sense: **exrex** generates a variable-count quantifier at a random length above its stated minimum, so whether the regex engine’s backtracking finds a valid truncated match depends on how much slack that length gives it and on the character immediately before the truncation point. A fixed-count rule offers no such slack, which is precisely why those two rules are exactly deterministic and these three are not. We report the single seeded run as the paper’s reproducible reference value throughout, and this range as evidence that the qualitative finding — severe, mechanism-specific degradation, an order of magnitude below the  $\geq 0.9976$  baseline — does not depend on which draw was reported.

## 7.2 The three metrics, per rule

Table 5 reports *BR*, *RLP* and *SP* for the five rules on which they are not all 1.0000; the remaining nineteen rules score 1.0000 on all three. The table makes two points that the aggregate in Table 4 cannot.

First, *BR* and the preservation metrics are orthogonal, and the two catastrophic rules fail in opposite ways. SendGrid and GCP both have *BR* = 0.0000: neither rule ever fires on a hyphen-terminated credential, in any of the ten independently seeded repetitions of Section 7.1. But SendGrid has *RLP* = 1.0000 and GCP has *RLP* = 0.9116. SendGrid’s failures are *total misses*, so they leave the preservation denominator entirely — of the credentials it did detect, every one was labelled correctly. GCP’s failures are *rescued and downgraded*, so they stay in the denominator and depress the ratio. A single scalar cannot express both; two are needed, and which one moves tells the maintainer whether the fix is a detection problem or a labelling problem.

Second, *SP* and *RLP* are numerically identical here, for a reason worth stating explicitly rather than leaving as a coincidence. The OTHER outcome — a different named rule firing — has rate exactly 0.0000 in all 251 recorded cells, so the only substitute label available is the entropy fallback, whose severity is fixed at MEDIUM. A fallback rescue therefore preserves severity only for a rule that is itself MEDIUM, and no MEDIUM rule in this table is ever rescued. The two metrics would separate on a scanner with overlapping rules of equal severity; on this one they cannot. We report both because the distinction is a property of the metric definitions, not of this dataset, and a replication on another tool should not have to rediscover it.

## 8 RQ2: Mechanism and Marginal Probability

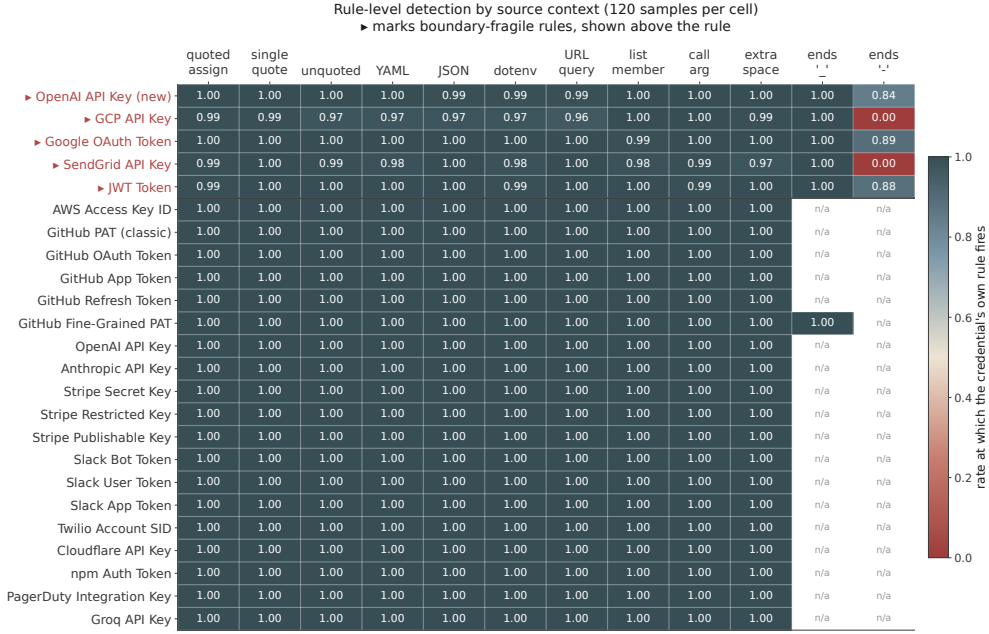
Five rules are boundary-fragile: *OpenAI API Key (new)*, *GCP API Key*, *Google OAuth Token*, *SendGrid API Key*, and *JWT Token*.

**Mechanism 1: the anchor.** Each ends with `\b` immediately after a character class containing a hyphen, e.g.



**Table 5** Boundary robustness, rule-label preservation and severity preservation. Only the five rules scoring below 1.0000 are shown; the other nineteen score 1.0000 throughout.

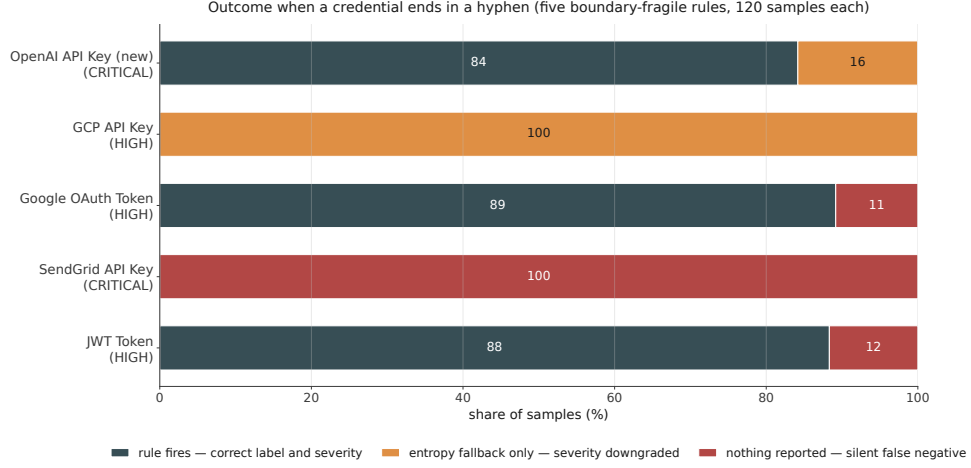
| Rule                     | Severity | <i>BR</i> | <i>RLP</i> | <i>SP</i> |
|--------------------------|----------|-----------|------------|-----------|
| SendGrid API Key         | CRITICAL | 0.0000    | 1.0000     | 1.0000    |
| GCP API Key              | HIGH     | 0.0000    | 0.9116     | 0.9116    |
| OpenAI API Key (new)     | CRITICAL | 0.8417    | 0.9861     | 0.9861    |
| JWT Token                | HIGH     | 0.8833    | 1.0000     | 1.0000    |
| Google OAuth Token       | HIGH     | 0.8917    | 1.0000     | 1.0000    |
| macro mean, all 24 rules |          | 0.9007    | 0.9957     | 0.9957    |



**Fig. 3** Rule-level detection across 24 rules and 12 contexts, 120 samples per cell. Blue indicates the credential’s own rule fired; red indicates it did not. “n/a” marks embeddings not applicable to that rule. A leading ▶ marks a boundary-fragile rule, listed above the robust rules and coloured red.

`r"\bSG\.[a-zA-Z0-9_-]{22}\.[a-zA-Z0-9_-]{43}\b"`

A hyphen is not a word character. When the final generated character is a hyphen and the following text character is also non-word — a closing quote, in the canonical case — no word boundary exists there and the match fails before any suppression logic runs. We confirmed that in failing samples every suppression predicate returns **False**; the scanner’s own heuristics judge the value to be a credential, yet nothing is reported.



**Fig. 4** Outcome decomposition for the five boundary-fragile rules when the credential ends in a hyphen. The two rules with  $BR = 0$  fail in opposite ways: SendGrid is never reported at all, while GCP is always reported but always downgraded. A tool-level metric would score GCP and OpenAI as fully detected.

**Mechanism 2: quantifier type.** The failure is not uniform, and the split is exact (Fig. 5a): fixed-count quantifiers admit no backtracking and fail totally, while variable-count quantifiers backtrack and match one character less.

| Rule                 | Quantifier     | Rule fires |
|----------------------|----------------|------------|
| SendGrid API Key     | {43} fixed     | 0.0%       |
| GCP API Key          | {35} fixed     | 0.0%       |
| OpenAI API Key (new) | {50,} variable | 84.2%      |
| JWT Token            | {10,} variable | 88.3%      |
| Google OAuth Token   | {68,} variable | 89.2%      |

The variable case is not benign: the rule reports a *truncated* credential. We isolated this on a minimal pair — `\bX[a-zA-Z0-9_]{10}\b` fails on a hyphen-terminated value while `\bX[a-zA-Z0-9_]{5,}\b` succeeds by matching one character fewer.

**Mechanism 3: why some failures are silent.** The entropy fallback captures candidates with `['\"]([a-zA-Z0-9+/_-]{20,})['\"]`, a class excluding the dot. Credentials containing a dot (SendGrid, Google OAuth, JWT) cannot be rescued and become silent misses; dotless ones (GCP, OpenAI) are rescued but relabelled from CRITICAL/HIGH to MEDIUM (Fig. 4). This is the distinction  $RLP(\rho)$  and  $SP(\rho)$  are defined to capture, and it is why GCP’s preservation score is the lowest in Table 5 despite its detection failures being the *least* silent of the five.

**Table 6** Marginal probability that a credential is not correctly labelled

| Credential type    | $P("-")$ | Marginal | Frequency             |
|--------------------|----------|----------|-----------------------|
| SendGrid API Key   | 1/64     | 0.015625 | 1 in 64 (silent miss) |
| GCP API Key        | 1/64     | 0.015625 | 1 in 64 (downgraded)  |
| Google OAuth Token | 1/64     | 0.001692 | 1 in 591              |
| JWT, HS384 (48 B)  | 1/64     | 0.001823 | 1 in 548              |
| JWT, HS256 (32 B)  | 0        | 0.000000 | impossible            |

## 8.1 Marginal, not conditional, probability

The rates above are *conditional* on the credential ending in a hyphen. The marginal rate is

$$P(\text{fail}) = P(\text{ends in "-"}) \times P(\text{fail} \mid \text{ends in "-"}),$$

and  $P(\text{ends in "-"})$  is *not* 1/64 for every type. Two cases must be separated.

For types whose tail is a uniform draw over the 64-symbol base64url-style alphabet (SendGrid, GCP, OpenAI, Google OAuth),  $P(\text{ends in "-"}) = 1/64$  exactly, since the alphabet contains exactly one hyphen symbol and every symbol is equally likely; we report this exact value rather than a simulated one, and confirm it is not being misapplied with five independent  $2 \times 10^5$ -draw Monte Carlo checks from a cryptographic generator (0.015555–0.015825 across the five checks, consistent with  $1/64 = 0.015625$  to within sampling error).

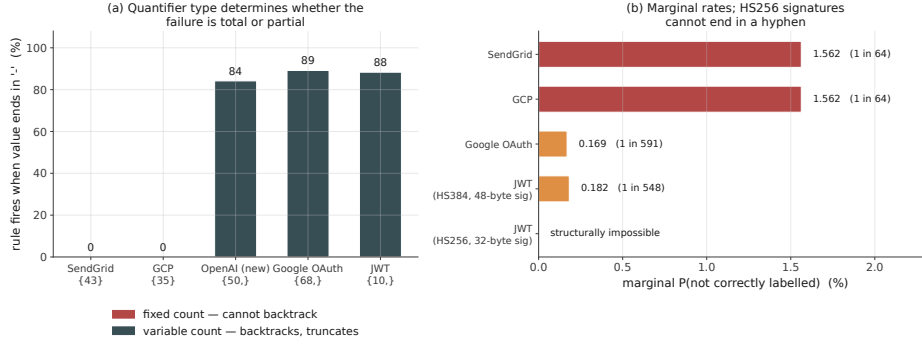
For JSON Web Tokens the tail is a base64url *encoding* of a signature of  $L$  bytes, and the final character carries only the residual bits: 6 bits when  $L \equiv 0 \pmod{3}$ , 4 bits when  $L \equiv 2$ , and 2 bits when  $L \equiv 1$ . The character “-” has base64url index 62, which is not a multiple of 4 or 16, so it is reachable *only* when  $L \equiv 0 \pmod{3}$ . HS256 uses a 32-byte signature ( $32 \bmod 3 = 2$ ), and we confirm empirically that only 16 distinct final characters occur and “-” never does. **An HS256 JWT is therefore structurally immune to this defect.** HS384 (48 bytes,  $\equiv 0$ ) is not.

Table 6 gives the corrected marginal rates. The practically consequential case is SendGrid, at exactly one credential in 64; the JWT case, which an earlier draft of this work overstated, is either impossible or roughly one in 548 depending on the signature algorithm.

The remedy is small and local: replace the trailing `\b` with a negative lookahead, `(?![A-Za-z0-9_])`, in the five affected rules. We do not merely propose this — Section 9 applies it, re-runs the full battery, and reports a regression we found in a plausible first attempt at the same fix.

## 9 Automated Repair and Fix Validation

Diagnosing a defect and proposing a remedy is not evidence that the remedy works. This section applies two candidate repairs to the five affected rules, in memory, without modifying the subject repository, and re-runs the full boundary-mutation battery against each.



**Fig. 5** (a) Quantifier type determines whether the failure is total or partial. (b) Corrected marginal rates; the encoding structure of HS256 signatures makes the defect unreachable for them.

## 9.1 Two candidates

Both replace the trailing `\b` that Section 8 identifies as the fault:

- **Candidate A**, `(?=[^\w-]|$)`, our first attempt: require the following character to be neither a word character nor a hyphen.
- **Candidate B**, `(?![A-Za-z0-9_])`, the recommended fix: require only that the following character is not a word character, which is what `\b` itself requires and is the minimal change that removes the hyphen special case.

Both are applied to exactly the five rules of Table 5, holding every other rule and the entropy fallback unchanged.

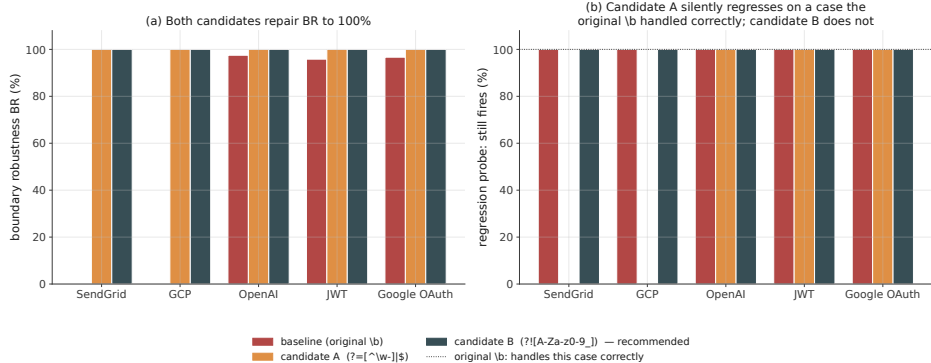
## 9.2 Both candidates repair the defect

Fig. 6a re-runs the full boundary-mutation battery under each candidate. Both restore  $BR = 1.0$  on all five rules: the hyphen-terminated case that previously failed at 0–89% now fires at 100% under either fix. Neither candidate introduces a single new finding on the 292,527-line real-code corpus of Section 12: 67 findings under the baseline rule table, and 67 under each candidate, file for file.

## 9.3 Candidate A regresses; Candidate B does not

A repair to an anchoring assertion can fail in a direction the boundary battery does not by itself probe: it can become *stricter* than the assertion it replaces, silently rejecting inputs the original correctly accepted. We therefore added one further probe, run under each candidate: a credential whose *own* final character is an ordinary word character, immediately followed by a hyphen in the surrounding text (for example, a credential embedded as `cfg_value = "...X-suffix"`). The original `\b` accepts this case — the boundary between a word character and a hyphen is a valid word boundary — so a correct repair must continue to accept it.

Candidate A does not. Fig. 6b shows it regressing to total failure (0%) on exactly the two rules governed by fixed-count quantifiers, SendGrid and GCP, on precisely this case — the same failure mode Section 8 diagnosed, reintroduced by the fix. The cause is



**Fig. 6** Fix validation. (a) Both candidates repair boundary robustness to 1.0 on all five affected rules. (b) A regression probe — a credential ending in a word character, followed by a hyphen — that the original rule handled correctly. Candidate A silently fails it on the two fixed-count-quantifier rules; Candidate B does not.

the same construct that caused the original defect:  $(?=[^\w-]|$)$  forbids a hyphen in the following text unconditionally, including here, where a hyphen legitimately follows a word-character-terminated credential. Candidate B imposes no such restriction and fires at 100% on all five rules under this probe, matching the original rule’s behaviour on the case it already handled correctly.

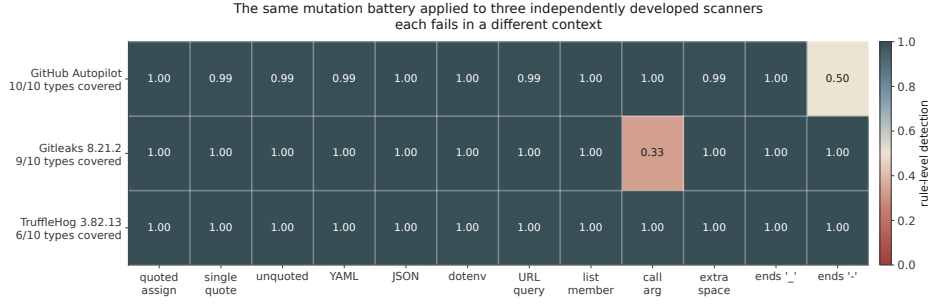
This is the paper’s second methodological point, not only its first result: a plausible one-line regex repair, chosen by inspection and applied to exactly the rules it was meant to fix, introduced a new defect of the same kind it removed. We found this only because we ran an oracle-based probe against the fix rather than reading it. We recommend Candidate B,  $(?![A-Za-z0-9_])$ , and report both results because a method for finding hidden assumptions in a detection rule should be applied to its own proposed repair, not only to the original.

## 10 RQ3: Does the Method Transfer?

We applied the identical battery to Gitleaks and TruffleHog, over 4,320 files. For external tools the expected rule per credential type is determined *empirically* from each tool’s own behaviour in the canonical context — whichever `RuleID` or `DetectorName` it fires there — so no tool is judged against another project’s naming. Types a tool never fires on are excluded from its aggregate and reported separately, so coverage gaps are not scored as fragility.

Fig. 7 shows the result: **the method transfers, and exposes a distinct behavioural profile in each tool**. Two of the three degrade, and they degrade in different contexts; the third does not degrade at all, which is itself a finding the same battery establishes.

**A second, unrelated defect in Gitleaks.** Gitleaks is unaffected by the trailing hyphen, but its rule-level detection falls to 0.333 when a credential appears as a bare call argument. Per-type inspection shows this is not uniform: `github-pat`, `github-oauth` and `slack-bot-token` are unaffected, while `aws-access-token`,



**Fig. 7** The same mutation battery applied to three independently developed scanners. The battery reveals scanner-specific degradation patterns; no tested context degraded all three scanners.

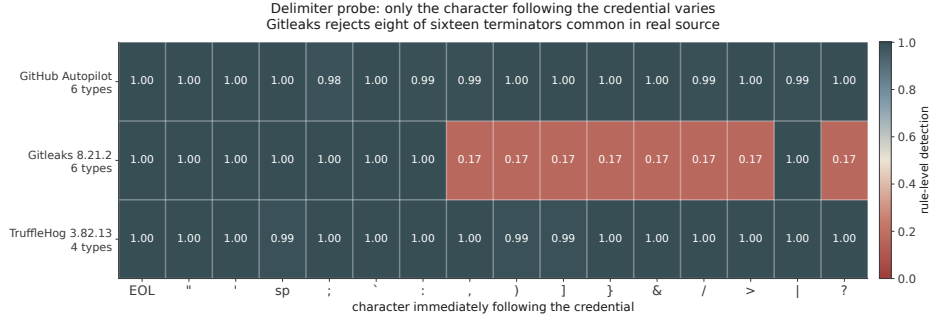
`gcp-api-key`, `npm-access-token`, `stripe-access-token`, `sendgrid-api-token` and `jwt` all drop to 0.000 — total misses.

To isolate the mechanism we ran a delimiter probe varying *only* the single character following the credential (Fig. 8). Gitleaks accepts end-of-line, both quote characters, whitespace, semicolon, backtick and colon, and rejects comma, closing parenthesis, closing bracket, closing brace, ampersand, forward slash, angle bracket and question mark. We confirmed the mechanism directly from source: Gitleaks’ public configuration, `config/gitleaks.toml` at tag `v8.21.2`, defines each affected rule (`aws-access-token`, `gcp-api-key`, `npm-access-token`, `sendgrid-api-token`, `stripe-access-token`, `jwt`) with an identical trailing non-capturing group appended to the credential-capturing group, `(?:[!'"|\\n|\\r|\\s|\\x60|;]|$)`, a Go regular-expression character class containing only the literal characters `,`, `|`, `"`, newline, carriage return, whitespace, backtick, and semicolon, or end-of-string. Every terminator we found rejected is provably absent from this class; every terminator we found accepted is provably present in it. The three unaffected types — `github-pat`, `github-oauth`, `slack-bot-token` — carry no such trailing group in their source regex at all, which is the source-level reason our data found them robust. We additionally verified the effect on minimal hand-written cases outside the harness: `cfg = "AKIA..."` is detected, `f(AKIA...)` is not.

The security relevance is direct. Every rejected character occurs routinely where credentials appear: `f(token)` in code, `[tok, x]` in a list, and — most consequentially — `?key=SECRET&next=1` in a URL, which is among the more common shapes in which credentials actually leak.

**TruffleHog shows no boundary fragility.** It is robust across every context and terminator, on the types it covers — but it covers the fewest (6 of 10 canonically). Its detectors are co-occurrence- and verification-oriented rather than shape-oriented, which is exactly the trade-off Section 11 quantifies.

**Taxonomy.** Three boundary-handling strategies, three failure profiles: an *implicit* anchor (`\\b`) yields probabilistic failure depending on the credential’s own final character; an *explicit* terminator allowlist yields deterministic failure depending on the following character; and *no* boundary dependence yields robustness at the cost of coverage. The method exposes all three; the defect class is tool-specific, the vulnerability to boundary assumptions is not.



**Fig. 8** Delimiter probe: only the character immediately following the credential varies. Gitleaks rejects eight of sixteen terminators that occur routinely in real source.

**Table 7** Three scanners on identical corpora (400 positives, 700 negatives), with Wilson 95% intervals. No value is emphasised: Section 11.1 shows that none of the between-tool differences is statistically significant at the correct unit of analysis.

| Corpus         | Tool               | Precision [95% CI]   | Recall [95% CI]      | F1    | TP / FP / FN  |
|----------------|--------------------|----------------------|----------------------|-------|---------------|
| A: format-spec | GitHub Autopilot   | 0.959 [0.936, 0.974] | 1.000 [0.991, 1.000] | 0.979 | 400 / 17 / 0  |
|                | Gitleaks 8.21.2    | 1.000 [0.988, 1.000] | 0.800 [0.758, 0.836] | 0.889 | 320 / 0 / 80  |
|                | TruffleHog 3.82.13 | 1.000 [0.984, 1.000] | 0.598 [0.549, 0.644] | 0.748 | 239 / 0 / 161 |
| B: structural  | GitHub Autopilot   | 0.959 [0.935, 0.974] | 0.998 [0.986, 1.000] | 0.978 | 399 / 17 / 1  |
|                | Gitleaks 8.21.2    | 1.000 [0.989, 1.000] | 0.900 [0.867, 0.926] | 0.947 | 360 / 0 / 40  |
|                | TruffleHog 3.82.13 | 1.000 [0.984, 1.000] | 0.600 [0.551, 0.647] | 0.750 | 240 / 0 / 160 |

## 11 RQ4: Comparison, Corpus, and Deployment Condition

Corpus construction alone moves Gitleaks’ recall from 0.800 to 0.900: its JWT rule validates internal token structure and correctly refuses a “JWT” whose segments are random characters. The subject’s scanner is almost unmoved, 1.000 to 0.998 — not because it is more capable, but because its rules are shallower (prefix, class, length). That shallowness also produces its 17 false positives where the others have none.

### 11.1 The comparison supports no ranking

The intervals in Table 7 are computed over samples, and taken at face value they do not overlap. That would be the wrong conclusion to draw, for a reason visible in the raw data: *per-type detection is very nearly deterministic*. Across both corpora and all three tools, fifty-eight of the sixty per-type recall values are exactly 0.000 or exactly 1.000. The two exceptions are both 0.975, a single miss in forty: TruffleHog on Slack bot tokens in corpus A, and the subject on SendGrid keys in corpus B. A tool either detects a credential type or it does not, with these two borderline exceptions; the 40 samples within a type are very close to 40 repetitions of one outcome rather than 40 independent observations.

**Table 8** Exact paired sign test over  $n=10$  credential types. “Discordant” counts types detected by the first tool and not the second, and conversely.

| Corpus | Comparison               | Discordant | $p$   |
|--------|--------------------------|------------|-------|
| A      | Autopilot vs. Gitleaks   | 2 / 0      | 0.500 |
|        | Autopilot vs. TruffleHog | 4 / 0      | 0.125 |
|        | Gitleaks vs. TruffleHog  | 2 / 0      | 0.500 |
| B      | Autopilot vs. Gitleaks   | 1 / 0      | 1.000 |
|        | Autopilot vs. TruffleHog | 4 / 0      | 0.125 |
|        | Gitleaks vs. TruffleHog  | 3 / 0      | 0.250 |

We state this as a measured property rather than an assumption because we tested it. An earlier version of the comparative harness recorded a fixed seed in its output while drawing every credential from `secrets`, a CSPRNG that cannot be seeded; the corpus therefore differed on every run. Under one such draw all sixty values were exactly 0.000 or 1.000, which would have made the determinism premise look exact when it is not. With the seed actually applied and the corpus fixed, two cells sit at 0.975. The inflation argument below is unaffected — a factor of forty is not recovered by two samples — but the premise is near-determinism, not determinism, and we report it as such.

The effective unit of analysis is therefore the credential *type*, of which there are ten, and a sample-level test would overstate the evidence by a factor of 40. We accordingly compare tools with an exact paired sign test over the ten types, reporting the discordant types themselves rather than a test statistic alone (Table 8).

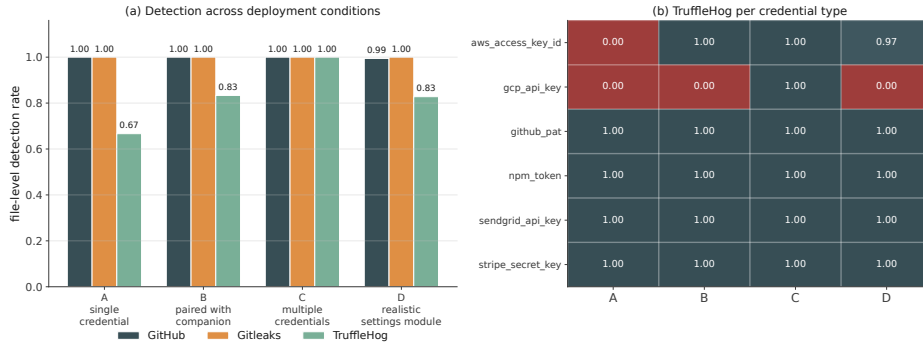
**No comparison reaches significance at  $\alpha = 0.05$** , and none comes close. The differences are entirely one-directional — every discordant type favours the same tool, never the reverse — which is consistent with a real ordering, but with ten types the sign test cannot resolve one: even a perfect 4/0 split has  $p = 0.125$ , because  $2^{-4}$  doubled is the smallest two-sided  $p$ -value ten paired types can produce at that margin. The design is underpowered for ranking by construction, and no larger sample of *credentials* would fix it; only more credential *types* would.

We therefore report Table 7 as a characterisation of where the tools differ and why, and we draw no ranking from it. This bears directly on the competing interest declared in Section 5.6: the raw numbers favour the author’s own tool, and the correct statistical treatment does not support that advantage. Readers should treat the coverage and trade-off structure below, not the recall ordering, as this section’s result.

**Deployment condition matters more.** Our single-credential-per-file design is adversarial to co-occurrence-based detection, so we evaluated four conditions (Fig. 9): (A) a credential alone; (B) paired with its companion secret; (C) several credentials in one file; (D) a realistic settings module with imports, comments and unrelated configuration.

TruffleHog’s detection rises from 0.667 (A) to 0.833 (B) to 1.000 (C), settling at 0.828 in the realistic condition (D). Per type, AWS goes from 0.000 in isolation to





**Fig. 9** Detection across four deployment conditions. TruffleHog’s recall depends strongly on corroborating context; the other two tools do not.

1.000 once a secret access key accompanies it. Its apparent recall deficit in Table 7 is therefore **substantially an artefact of the corpus**, not a capability limit, and the single-credential numbers should be read as a lower bound under conditions adversarial to that architecture. The other two tools are near-invariant across conditions.

## 12 RQ5: False Positives on the Evaluated Real-Code Corpus

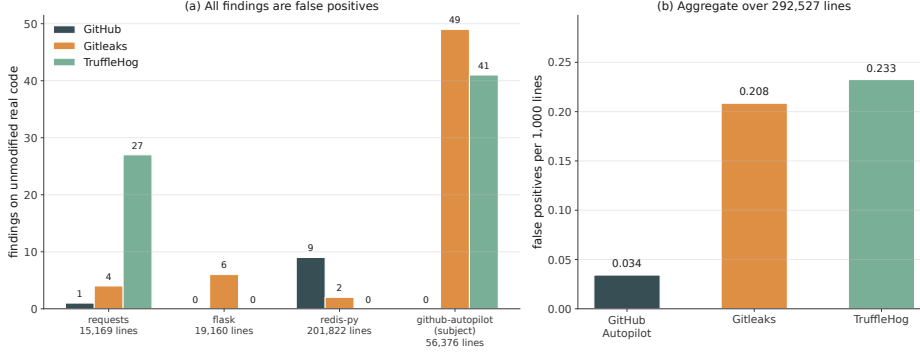
We scanned 292,527 lines of unmodified source from four open-source projects — `requests`, `flask`, `redis-py`, and the subject — line by line as added diff lines, mirroring the subject’s push handler. All four are pinned by commit in the replication package.

This corpus is small and Python-centric, and it is not a sample of anything. We therefore make no claim about real-world false-positive rates in general; the claim is about *these four projects*, and its purpose is to test whether a conclusion drawn from the synthetic corpus survives contact with unmodified code. It does not.

The ordering *inverts* relative to the synthetic corpus (Fig. 10). On constructed negatives the subject’s scanner had the worst precision; on this corpus it has the lowest rate: 0.034 findings per kLOC against Gitleaks’ 0.208 and TruffleHog’s 0.233. The synthetic corpus over-represented bare high-entropy base64 assignments, to which its unanchored fallback is uniquely vulnerable, and under-represented the documentation and configuration idioms that dominate real repositories.

We inspected all ten of its findings; every one is a false positive, in three groups: six occurrences of a documentation URL *template* (`redis://[[username] : [password]] @localhost:6379/0`) matched at CRITICAL severity; three doctest passwords; and one RFC 3986 unreserved-character constant matched by the entropy fallback.

For fairness: 49 of Gitleaks’ and 41 of TruffleHog’s findings occur in the subject’s own repository, which contains credential-shaped strings in its rule table and fixtures and which its scanner suppresses via path rules tuned for that layout. Restricted to the three third-party projects the totals are 10, 12, and 27.



**Fig. 10** False positives on the four unmodified open-source projects evaluated. The ordering inverts relative to the synthetic corpus.

## 13 Discussion

### 13.1 Example-based validation has a blind spot with a shape

All five defective rules in the primary subject, and all six in Gitleaks, pass their projects’ own test suites, because every fixture places the credential in a canonical position. The blind spot is not random: it is exactly the set of behaviours that depend on the credential’s boundary, and it is invisible to any suite whose fixtures share one embedding. Neither defect we found is exotic — one fires on roughly one SendGrid key in 63, the other on any credential followed by a comma. The battery that finds both is small and cheap: twelve embeddings generated from the rules themselves, running in under twenty seconds over 24 rules. The cost of *not* running it is a silent false negative in a security control.

### 13.2 Rule-level oracles are not optional

Measuring only whether something was reported would have scored GCP and OpenAI keys as fully detected while their severity was being silently downgraded from CRITICAL or HIGH to MEDIUM. It would also have concealed our own harness defect (Section 5.4), which a tool-level oracle could not have distinguished from correct behaviour.  $RLP(\rho)$  and  $SP(\rho)$  exist to make that distinction quantitative, and Table 5 shows why one number is not enough: two rules with identical, maximally bad  $BR$  scores differ completely in whether their failures are silent or merely mislabelled, which is the difference between an undetected leak and a deprioritised alert.

### 13.3 Boundary strategy is a design axis with real consequences

The three scanners embody three distinct strategies, and the method surfaces the cost of each. An implicit anchor fails probabilistically, as a function of the credential’s own final character. An explicit terminator allowlist fails deterministically, as a function of the character that follows. Boundary-independent detection avoids both failure modes but covers the fewest credential shapes. No scanner dominates across all evaluated

dimensions and conditions — and, as Section 11.1 establishes, our design is not powered to rank them even where it might appear to. What the method delivers is not a winner but a diagnosis: for a given rule table, which of these three failure modes applies, and to which rules.

### 13.4 Benchmark construction, not only tool quality, decides what a comparison reports

Three separate results here are warnings about our own methodology. Corpus construction moves Gitleaks by +0.10 recall. Deployment condition moves TruffleHog by +0.33 detection. Synthetic versus real negatives invert the precision ordering entirely. A single-corpus, single-condition evaluation of these three tools would have produced a confident conclusion in any of three mutually contradictory directions, depending only on which condition the experimenter happened to pick. We take this as an argument for reporting the sensitivity of a comparison to its own construction as a first-class result, rather than reporting a single table and a ranking.

## 14 Threats to Validity

### 14.1 Construct validity

Twelve embeddings do not exhaust the boundary space. One family we know is missing is encoding transformations — base64-wrapped configuration, escaped strings, template interpolation — which are untested and plausibly hide a further defect family; Section 15 treats this as the first extension.

Positive corpora are generated rather than harvested, because real leaked credentials cannot be redistributed. As Section 5.1 sets out, this guarantees syntactic and not semantic validity: a defect found on a string the provider would never issue may have no field consequence. Sections 8–10 do not depend on the distinction, since generation is from the rules under test and the finding is an internal inconsistency in the rule. Section 11 does depend on it, and quantifies how much.

The marginal probabilities in Table 6 assume a uniform final character within each type’s alphabet;  $P(“-”) = 1/64$  then follows exactly, and we report that exact value rather than a simulated one, confirming it independently by Monte Carlo rather than by construction alone. Real provider issuance may deviate from uniformity, in which case the rates shift; the JWT rows depend additionally on signature length, which we enumerate rather than assume.

The three variable-quantifier rules in Table 5 carry a construct-validity subtlety the two fixed-quantifier rules do not: their truncated-match rate is itself a random variable, not a fixed property of the rule. **exrex** generates a length above each quantifier’s stated minimum, and whether backtracking finds a valid truncated match depends on how much slack that length gives it. Section 7.1 reports both the single reproducible value we use throughout and the range across ten independent repetitions, and explains why only these three rules show it.

Finally, *SP* requires a severity oracle, and we have one only for the primary subject. Section 10 therefore reports *BR* and *RLP* for the external scanners and no *SP*, rather than constructing a severity correspondence the tools do not share.

## 14.2 Internal validity

Three harness defects were found during this work and are reported rather than silently corrected. First, an identifier that triggered the subject’s generic keyword rules and masked the defect entirely (Section 5.4); this is documented in full because it is the failure mode a replication is most likely to reproduce. Second, an unsynchronised thread start in the supplementary systems benchmark (Online Resource 1, Section 1). Third, the original mutation harness recorded a fixed seed value in its output metadata without ever calling the corresponding seeding function, so the figures in an earlier draft of this paper were not actually reproducible from that seed despite claiming to be: re-running the unseeded harness a second time reproduced neither the recorded rates nor the recorded *count* of boundary-fragile rules (three rather than five). We added the missing seeding call, confirmed exact reproducibility by re-running the corrected harness and diffing every field but wall-clock time against a second run, and used that genuinely seeded output throughout this paper. We disclose this because a paper that argues for validating claims empirically should not exempt its own infrastructure from the same standard.

The Gitleaks mechanism is confirmed from its published source configuration (Section 10), quoting the exact character class. The TruffleHog co-occurrence mechanism is behavioural only, established by direct probing rather than by reading its Go source, and we mark it as such. TruffleHog is run with `--no-verification`; live verification would change its behaviour and was not available in our environment.

The expected-rule freezing procedure of Section 5.3 cannot detect an attribution error that is both consistent across canonical samples and semantically plausible. We checked the frozen identifiers by inspection but cannot rule this out mechanically.

## 14.3 External validity

The evaluation covers one hardware configuration and one Python version. The real-code corpus is four Python-centric projects pinned by commit, which is not a sample of GitHub and supports no general claim about false-positive rates in the field (Section 12). Cross-tool results cover ten shared credential types, not any tool’s full rule table; extending to full tables would convert our taxonomy into a census, which we have not done.

## 14.4 Conclusion validity

The most important limitation is the one in Section 11.1: per-type detection is near-deterministic, so the effective sample size for between-tool comparison is ten credential types, not 400 credentials. At that size no pairwise difference reaches significance, and we draw no ranking. Readers should not read Table 7’s intervals as licensing one.

TruffleHog’s figures in that table are a lower bound under conditions adversarial to its architecture, established by direct probing and quantified in Fig. 9. Real-code

ground truth assumes released public code contains no live credentials; we inspected every finding individually, but an undetected genuine leak would shift the counts.

## 15 Future Work

Five directions follow, ordered by how much they would strengthen the method rather than by effort. First, extend the battery beyond boundary embeddings to encoding transformations, the family we expect most likely to conceal a second defect class. Second, apply the method to the full rule tables of both external scanners rather than ten shared types, converting the taxonomy of Section 10 into a census and, incidentally, supplying the credential types that Section 11.1 shows are the binding constraint on any statistical comparison. Third, enlarge the real-code corpus with an explicit, stated sampling frame across languages, so that false-positive behaviour can be characterised rather than merely observed on four projects. Fourth, obtain severity oracles for the external scanners, which would allow *SP* to be reported across all three. Fifth, extend the repair-and-validate loop of Section 9 to Gitleaks’ terminator-allowlist defect, which we have diagnosed but not patched; we would expect the same lesson to recur, that a plausible one-line fix is a hypothesis until it is run against the battery that found the original defect.

## 16 Conclusion

Pattern-based secret scanners are validated by example. Example-based fixtures hold constant the one variable that decides whether an anchored regular expression matches: the text surrounding the credential. Boundary-mutation testing varies it systematically. Credentials are generated from each rule’s own expression and embedded in contexts that real credentials occupy. Outcomes are then judged at the rule level rather than the tool level. This yields three metrics, which separate failing to detect a credential from detecting it and labelling it wrongly.

Applied to three independently developed scanners, the method transferred to all three. It exposed a distinct behavioural profile in each. In the first, a hyphen-terminated credential defeats an implicit word-boundary anchor. In the second, eight routine punctuation characters defeat an explicit terminator allowlist, confirmed from source. The third shows no boundary fragility at all, at the cost of the narrowest coverage and a strong dependence on corroborating context. Each defect is reported with its mechanism rather than as a symptom. Each is also reported with the marginal, not merely conditional, probability that it affects a real credential — an analysis that shows one widely deployed credential format to be structurally immune and another to fail once in 64.

We do not stop at diagnosis. We patch the primary subject’s five defective rules, re-run the full battery, and confirm the repair with no regression and no new false positives on real code. We report, alongside that success, a repair candidate that looked equally plausible and was not: an inspection-only fix silently reintroduced the same failure mode on two of the five rules, caught only because we ran the same oracle-based probe against our own patch that we ran against the original defect.

For practice, the implication is twofold: a rule table can be tested against itself, cheaply, finding defects the project’s own suite is structurally incapable of observing; and a proposed fix to such a defect is a hypothesis, not a fact, until it is tested the same way. For evaluation, the implication is more cautionary. In three separate places our own benchmark construction could have reversed the conclusion. At the correct unit of analysis, our comparison cannot rank the three tools at all. We report that negative result alongside the positive ones. A method for exposing hidden assumptions in rule-based detection tests, and for validating proposed repairs to them, is worth little if it is not turned on the evaluation that carries it.

## Statements and Declarations

**Funding.** This research received no external funding.

**Competing interests.** The author is the creator and maintainer of GitHub Autopilot, one of the three systems evaluated in this study. To mitigate the resulting researcher bias, the evaluation used a subject version frozen before any measurement, pinned and checksum-verified external tool binaries, byte-identical corpora and execution conditions across all three tools, and retention of all raw outputs including those unfavourable to the subject. No result was filtered after inspection and no defect in the subject was repaired before measurement. Section 5.6 states these controls in full and identifies the specific findings that are unfavourable to the subject, including the paired analysis in Section 11.1, which withdraws the recall advantage the raw numbers would otherwise give it. The author has no financial interest in, and no relationship with, the maintainers of Gitleaks or TruffleHog.

**Data availability.** All raw experimental outputs — per-rule mutation results, cross-tool and delimiter probes, comparative and co-occurrence benchmarks, real-code findings, and the probability analysis — are included in the replication package as unmodified JSON, together with the redacted finding logs. The complete replication package is archived on Zenodo at <https://doi.org/10.5281/zenodo.22114221> (Mishra 2026) and is also browsable in the `research/` directory of the subject repository.

**Code availability.** All benchmark harnesses, the figure-generation code, a pinned environment specification, and a one-command `reproduce.sh` are included in the replication package, which is archived on Zenodo and also browsable in the subject repository under the `research/` directory at <https://github.com/Shweta-Mishra-ai/github-autopilot/tree/main/research>. The subject system itself is public at <https://github.com/Shweta-Mishra-ai/github-autopilot> and pinned at commit 38b2013; the comparison tools are official release binaries at versions 8.21.2 and 3.82.13, verified by SHA-256 at reproduction time.

**Author contributions.** Shweta Mishra is the sole author and is responsible for the study design, implementation, execution, analysis, and manuscript.

**Ethical approval.** Not applicable. This study involved no human participants and no personal data. All credentials used are synthetic, generated from published format specifications; no live or leaked credential was collected, tested, or redistributed.

**Consent to participate.** Not applicable. This study involved no human participants.

**Consent to publish.** Not applicable. This study contains no data, images, or details relating to an individual person.

**Supplementary Information. Online Resource 1** (ESM.1.pdf): operational characterisation of the primary subject — lost-update correctness oracle, synchronised-start concurrency scaling from 1 to 128 threads, ingress latency against payload size, fault-injection and circuit-breaker behaviour, and test-suite and complexity characteristics. None of it is evidence for or against any claim in the main paper.

**Replication Package.** The package contains all harnesses, raw JSON results, redacted finding logs, figure code, a pinned environment, and a one-command `reproduce.sh`, together with the complete raw data behind the operational characterisation of Online Resource 1. It resides in the `research/` directory of the subject repository, <https://github.com/Shweta-Mishra-ai/github-autopilot/tree/main/research>. The subject itself is pinned at commit `38b2013` of that repository, the real-code corpora at their scanned commits, and the comparison tools at Gitleaks 8.21.2 and TruffleHog 3.82.13 with recorded SHA-256 digests. An archived, versioned snapshot of the package is deposited on Zenodo at <https://doi.org/10.5281/zenodo.22114221> (Mishra 2026). No number in this paper was estimated or extrapolated; each traces to a recorded script output.

## References

- Basak SK, Neil L, Reaves B, et al (2022) What are the practices for secret management in software artifacts? In: Proc. IEEE SecDev, pp 69–76
- Basak SK, Cox J, Reaves B, et al (2023a) A comparative study of software secrets reporting by secret detection tools. In: Proc. ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM), pp 1–12, arXiv:2307.00714
- Basak SK, Neil L, Reaves B, et al (2023b) SecretBench: A dataset of software secrets. In: Proc. IEEE/ACM Int. Conf. Mining Software Repositories (MSR), Data and Tool Showcase Track, pp 347–351, <https://doi.org/10.1109/MSR59073.2023.00053>
- Chapman C, Stolee KT (2016) Exploring regular expression usage and context in Python. In: Proc. ISSTA, pp 282–293
- Chen TY, Kuo FC, Liu H, et al (2018) Metamorphic testing: A review of challenges and opportunities. ACM Comput Surv 51(1):1–27. <https://doi.org/10.1145/3143561>
- Claessen K, Hughes J (2000) QuickCheck: A lightweight tool for random testing of Haskell programs. In: Proc. ICFP, pp 268–279, <https://doi.org/10.1145/351240.351266>
- DeMillo RA, Lipton RJ, Sayward FG (1978) Hints on test data selection: Help for the practicing programmer. Computer 11(4):34–41. <https://doi.org/10.1109/C-M.1978.218136>

- Fowler M (2014) CircuitBreaker. <https://martinfowler.com/bliki/CircuitBreaker.html>
- Gitleaks contributors (2024) Gitleaks: a SAST tool for detecting secrets, v8.21.2. <https://github.com/gitleaks/gitleaks>
- Godefroid P, Kiezun A, Levin MY (2008) Grammar-based whitebox fuzzing. In: Proc. PLDI, pp 206–215
- Hardt D (2012) The OAuth 2.0 authorization framework. Tech. Rep. RFC 6749, IETF
- Herbold S, Trautsch A, Trautsch F (2020) On the feasibility of automated prediction of bug and non-bug issues. *Empirical Software Engineering* 25(6):5333–5369. <https://doi.org/10.1007/s10664-020-09885-w>
- Jia Y, Harman M (2011) An analysis and survey of the development of mutation testing. *IEEE Trans Softw Eng* 37(5):649–678. <https://doi.org/10.1109/TSE.2010.62>
- Jones M, Bradley J, Sakimura N (2015) JSON Web Token (JWT). Tech. Rep. RFC 7519, IETF
- Krawczyk H, Bellare M, Canetti R (1997) HMAC: Keyed-hashing for message authentication. Tech. Rep. RFC 2104, IETF
- Li K, Ling L, Yang J, et al (2025) Automatically detecting checked-in secrets in Android apps: How far are we? *Empirical Software Engineering* <https://doi.org/10.1007/s10664-025-10772-5>
- Liem CCS, Panichella A (2020) Run, forest, run? On randomization and reproducibility in predictive software engineering. *ArXiv:2012.08387*
- McCabe TJ (1976) A complexity measure. *IEEE Trans Softw Eng* SE-2(4):308–320. <https://doi.org/10.1109/TSE.1976.233837>
- Meli M, McNiece MR, Reaves B (2019) How bad can it git? Characterizing secret leakage in public GitHub repositories. In: Proc. NDSS
- Michael LG, Donohue J, Davis JC, et al (2019) Regexes are hard: Decision-making, difficulties, and risks in programming regular expressions. In: Proc. ASE, pp 415–426, <https://doi.org/10.1109/ASE.2019.00047>
- Mishra S (2026) Boundary-mutation testing for pattern-based secret detection: Replication package. <https://doi.org/10.5281/zenodo.22114221>
- Nygaard MT (2018) Release It!, 2nd edn. Pragmatic Bookshelf
- Rahman A, Parnin C, Williams L (2019) The seven sins: Security smells in Infrastructure as Code scripts. In: Proc. ICSE, pp 164–175



- Saha A, Denning T, Srikumar V, et al (2020) Secrets in source code: Reducing false positives using machine learning. In: Proc. COMSNETS, pp 168–175
- Segura S, Fraser G, Sanchez AB, et al (2016) A survey on metamorphic testing. IEEE Trans Softw Eng 42(9):805–824
- Truffle Security Co. (2024) TruffleHog, v3.82.13. <https://github.com/trufflesecurity/trufflehog>
- Vasilescu B, Yu Y, Wang H, et al (2015) Quality and productivity outcomes relating to continuous integration in GitHub. In: Proc. ESEC/FSE, pp 805–816
- Wessel M, de Souza BM, Steinmacher I, et al (2018) The power of bots: Characterizing and understanding bots in OSS projects. Proc ACM Hum-Comput Interact 2(CSCW):182:1–182:19
- Zhu H, Hall PAV, May JHR (1997) Software unit test coverage and adequacy. ACM Comput Surv 29(4):366–427